Project Overview

The Chicago Police Department (CPD) receives crime data spanning multiple years but lacks a systematic way to analyze it. This project builds a Python-based data pipeline that ingests, cleans, stores, and analyzes Chicago crime records to help CPD identify crime trends, high-risk areas, and arrest patterns — enabling data-driven decisions on patrol allocation and crime prevention.

Pipeline Architecture

INGEST → CLEAN → STORE → ANALYZE → REPORT

CSVs	Pandas	SQLite DB	Pandas/ NumPy/SQL	Charts + Answers

Use Case 1 — Load & Clean

```
File  Edit  Format  Run  Options  Window  Help
import pandas as pd
import numpy as np
import sqlite3
import os
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "..", "data", "Chicago_Datasets_Python")
DB_PATH = os.path.join(BASE_DIR, "..", "data", "chicago_crime.db")
SCHEMA_PATH = os.path.join(BASE_DIR, "..", "sql", "schema_sqlite.sql")

if os.path.exists(DB_PATH):
    os.remove(DB_PATH)
    print("Old database removed.")

conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
with open(SCHEMA_PATH, "r") as f:
    schema_sql = f.read()
cursor.executescript(schema_sql)
conn.commit()
print("Schema created fresh.")

#Load
df = pd.read_csv(os.path.join(BASE_DIR, "..", "data", "Chicago_Datasets_Python", "chicago_crime_dataset.csv"))
print("\n1: LOAD DATASET ===")
print("Shape (rows, columns):", df.shape)
print("\nSchema / Data types:\n", df.dtypes)
print("\nFirst 10 rows:\n", df.head(10))
```

```
#Clean
print("\n2: CLEAN DATASET")
df['date'] = pd.to_datetime(df['date'], errors='coerce')
df['date_of_update'] = pd.to_datetime(df['date_of_update'], errors='coerce')
print("Date column converted. Missing dates after conversion:", df['date'].isna().sum())

print("\nMissing values (%) per column:\n", (df.isna().mean() * 100).round(2).sort_values(ascending=False))

df['location_desc'] = df['location_desc'].fillna('UNKNOWN')

for col in ['primary_type', 'description', 'location_desc', 'block']:
    df[col] = df[col].astype(str).str.strip().str.upper()
print("\nCategorical fields standardized (stripped + uppercased).")
print("Sample primary_type values:", df['primary_type'].unique()[:5])

#Feature engineering
print("\n3: FEATURE ENGINEERING")
df['crime_month'] = df['date'].dt.month
df['crime_dayofweek'] = df['date'].dt.day_name()
print(df[['date', 'crime_month', 'crime_dayofweek']].head())

#NumPy missing value check
print("\n4: NUMPY MISSING VALUE ANALYSIS")
missing_pct = np.round(df.isna().mean().values * 100, 2)
cols_over_50 = df.columns[missing_pct > 50].tolist()
print("Columns with >50% missing:", cols_over_50 if cols_over_50 else "None — no columns dropped")

#Convert datetime to string for SQLite
df['date'] = df['date'].astype(str)
df['date_of_update'] = df['date_of_update'].astype(str)
```

```
#Insert lookup tables
print("\n5: INSERT INTO DATABASE")
iucr_df = pd.read_csv(os.path.join(DATA_DIR, "iucr_codes.csv"))
iucr_df.columns = iucr_df.columns.str.lower()
iucr_df.to_sql("iucr", conn, if_exists="append", index=False)

beat_df = pd.read_csv(os.path.join(DATA_DIR, "chicago_police_beat_info.csv"))
beat_df.columns = beat_df.columns.str.lower()
beat_df.to_sql("police_beat_info", conn, if_exists="append", index=False)

district_df = pd.read_csv(os.path.join(DATA_DIR, "chicago_district_ps_info.csv"))
district_df.columns = district_df.columns.str.lower()
district_df.to_sql("district_ps_info", conn, if_exists="append", index=False)

ward_df = pd.read_csv(os.path.join(DATA_DIR, "chicago_ward_offices.csv"))
ward_df.columns = ward_df.columns.str.lower()
ward_df.to_sql("ward_office", conn, if_exists="append", index=False)

community_df = pd.read_csv(os.path.join(DATA_DIR, "chicago_city_community.csv"))
community_df.columns = community_df.columns.str.lower()
community_df.to_sql("city_community", conn, if_exists="append", index=False)
print("Lookup tables loaded.")

df.to_sql("chicago_crime", conn, if_exists="append", index=False)
print("Crime table loaded:", len(df), "rows")

conn.close()

#Answer Use Case 1 questions
print("\nUSE CASE 1 — FINAL ANSWERS")
print("Unique crime types:", df['primary_type'].nunique())
print("Date range:", df['date'].min(), "to", df['date'].max())
print("Anomaly: Background states dataset covers 2012–2016, but actual range is",
      df['date'].min(), "to", df['date'].max(), "– significant mismatch.")
```

OUTPUT

```
Old database removed.
Schema created fresh.

1: LOAD DATASET ===
Shape (rows, columns): (2000, 22)

Schema / Data types:
 id                   int64
case_number          object
date                 object
block                object
iucr_code             int64
primary_type         object
description          object
location_desc        object
arrest                 bool
domestic               bool
beat_num              int64
district_code         int64
ward_no             float64
community_code      float64
fbi_code             object
x_coordinate        float64
y_coordinate        float64
year                   int64
date_of_update       object
latitude            float64
longitude           float64
location             object
dtype: object
```

```
First 10 rows:
          id case_number  ...  longitude                    location
0  10000000    SS442871  ... -87.673423  (41.71608165, -87.67342345)
1  10000001    WW348136  ... -87.626794  (41.94851207, -87.62679436)
2  10000002    GG525832  ... -87.629846  (41.96394084, -87.62984598)
3  10000003    NC685131  ... -87.607711  (41.77004756, -87.60771146)
4  10000004    YT673638  ... -87.602075  (41.72772815, -87.60207475)
5  10000005    CR348066  ... -87.582994  (41.85177018, -87.58299423)
6  10000006    XE039726  ... -87.522090  (41.98069888, -87.52208991)
7  10000007    QK559272  ... -87.621489  (41.74574749, -87.62148947)
8  10000008    YD115267  ... -87.709245  (41.85296539, -87.70924486)
9  10000009    DI878822  ... -87.516485  (41.72356216, -87.51648472)

[10 rows x 22 columns]
```

```
2: CLEAN DATASET
Date column converted. Missing dates after conversion: 0

Missing values (%) per column:
 location_desc     4.70
location           3.70
latitude           3.65
x_coordinate       3.35
longitude          3.20
y_coordinate       3.20
ward_no            1.85
community_code     1.80
primary_type       0.00
iucr_code          0.00
case_number        0.00
id                 0.00
block              0.00
date               0.00
beat_num           0.00
district_code      0.00
arrest             0.00
domestic           0.00
description        0.00
fbi_code           0.00
year               0.00
date_of_update     0.00
dtype: float64

Categorical fields standardized (stripped + uppercased).
Sample primary_type values: ['THEFT' 'PUBLIC PEACE VIOLATION' 'VANDALISM' 'MOTOR VEHICLE THEFT'
 'ASSAULT']
```

```
3: FEATURE ENGINEERING
                  date  crime_month crime_dayofweek
0 2022-03-04 07:47:00            3          Friday
1 2015-06-27 19:34:00            6        Saturday
2 2021-03-31 10:20:00            3       Wednesday
3 2019-04-26 13:29:00            4          Friday
4 2017-09-07 05:56:00            9        Thursday

4: NUMPY MISSING VALUE ANALYSIS
Columns with >50% missing: None — no columns dropped

5: INSERT INTO DATABASE
Lookup tables loaded.
Crime table loaded: 2000 rows

USE CASE 1 — FINAL ANSWERS
Unique crime types: 16
Date range: 2015-01-03 23:10:00 to 2023-12-31 09:44:00
Anomaly: Background states dataset covers 2012-2016, but actual range is 2015-01-03 23:10:00 to 2023-12-31 09:44:00 - significant mismatch.
```

Use Case 2 - EDA & Visualization

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "..", "data", "chicago_crime.db")
OUTPUTS_DIR = os.path.join(BASE_DIR, "..", "outputs")

os.makedirs(OUTPUTS_DIR, exist_ok=True)

conn = sqlite3.connect(DB_PATH)
df = pd.read_sql("SELECT * FROM chicago_crime", conn)
conn.close()

df['date'] = pd.to_datetime(df['date'])

#1: Crime Trend Over Years
print("1: CRIME TREND OVER YEARS")
yearly = df.groupby('year').size()
plt.figure(figsize=(10, 6))
yearly.plot(kind='line', marker='o', color='steelblue')
plt.title("Total Crimes Per Year")
plt.xlabel("Year")
plt.ylabel("Number of Crimes")
plt.grid(True)
plt.savefig(os.path.join(OUTPUTS_DIR, "07_crime_trend_yearly.png"))
plt.close()
print(yearly)


#2: Crime Distribution by Category
print("\n2: CRIME DISTRIBUTION BY CATEGORY")
top10_types = df['primary_type'].value_counts().head(10)
top10_pct = (top10_types / len(df) * 100).round(2)
plt.figure(figsize=(10, 6))
top10_types.plot(kind='bar', color='steelblue')
plt.title("Top 10 Crime Categories")
plt.xlabel("Primary Type")
plt.ylabel("Count")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig(os.path.join(OUTPUTS_DIR, "08_top10_crime_categories.png"))
plt.close()
print("Counts:\n", top10_types)
print("\nPercentages:\n", top10_pct)

#3: Arrests and Crime Outcomes
print("\n3: ARRESTS AND CRIME OUTCOMES")
arrest_rate = df['arrest'].mean() * 100
print(f"Overall arrest rate: {arrest_rate:.2f}%")
```

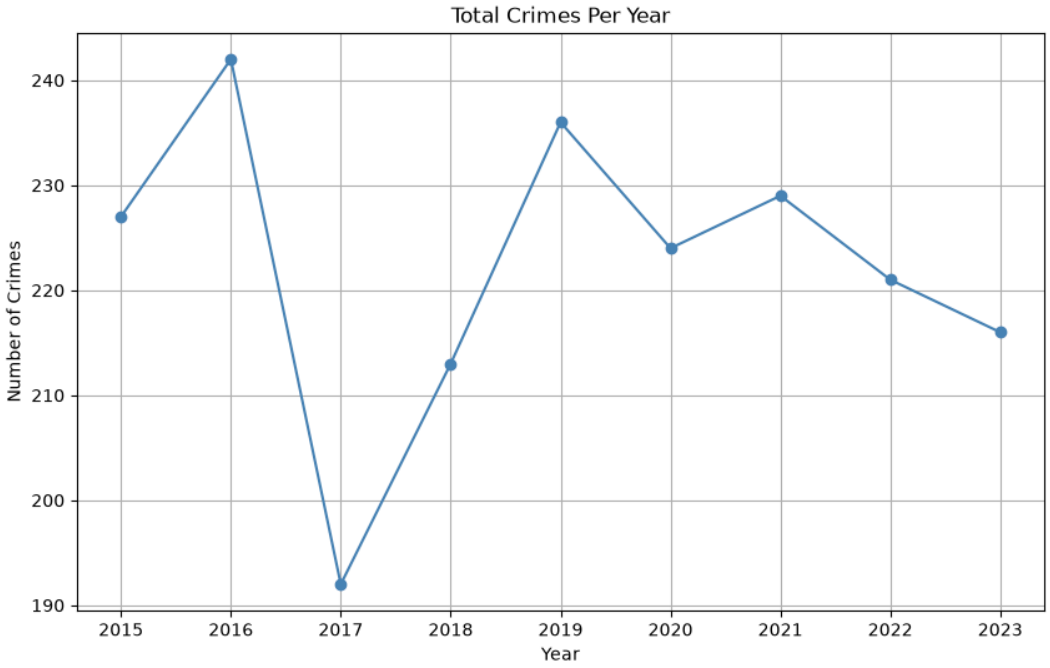
```
#4: Heatmap of Crime by Month and Day of Week
print("\n4: HEATMAP - MONTH VS DAY OF WEEK")
pivot = df.pivot_table(index='crime_month', columns='crime_dayofweek', values='id', aggfunc='count')
day_order = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday']
pivot = pivot[day_order]
plt.figure(figsize=(10, 8))
sns.heatmap(pivot, annot=True, fmt='.0f', cmap='YlOrRd')
plt.title("Crime Frequency: Month vs Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Month")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUTS_DIR, "09_heatmap_month_dayofweek.png"))
plt.close()
print("Heatmap saved.")

#5: Top Community Areas
print("\n5: TOP COMMUNITY AREAS")
top_areas = df['community_code'].value_counts().head(10)
plt.figure(figsize=(10, 6))
top_areas.plot(kind='bar', color='darkgreen')
plt.title("Top 10 Community Areas by Crime Count")
plt.xlabel("Community Code")
plt.ylabel("Crime Count")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUTS_DIR, "10_top10_community_areas.png"))
plt.close()
print(top_areas)

#Use Case 2 Questions
print("\nUSE CASE 2 — FINAL ANSWERS")
print("Most frequent crime category:", df['primary_type'].value_counts().idxmax())

arrest_by_year = (df.groupby('year')['arrest'].mean() * 100).round(2)
print("\nArrest rate by year (consistency check):\n", arrest_by_year)

month_freq = df.groupby('crime_month').size()
```

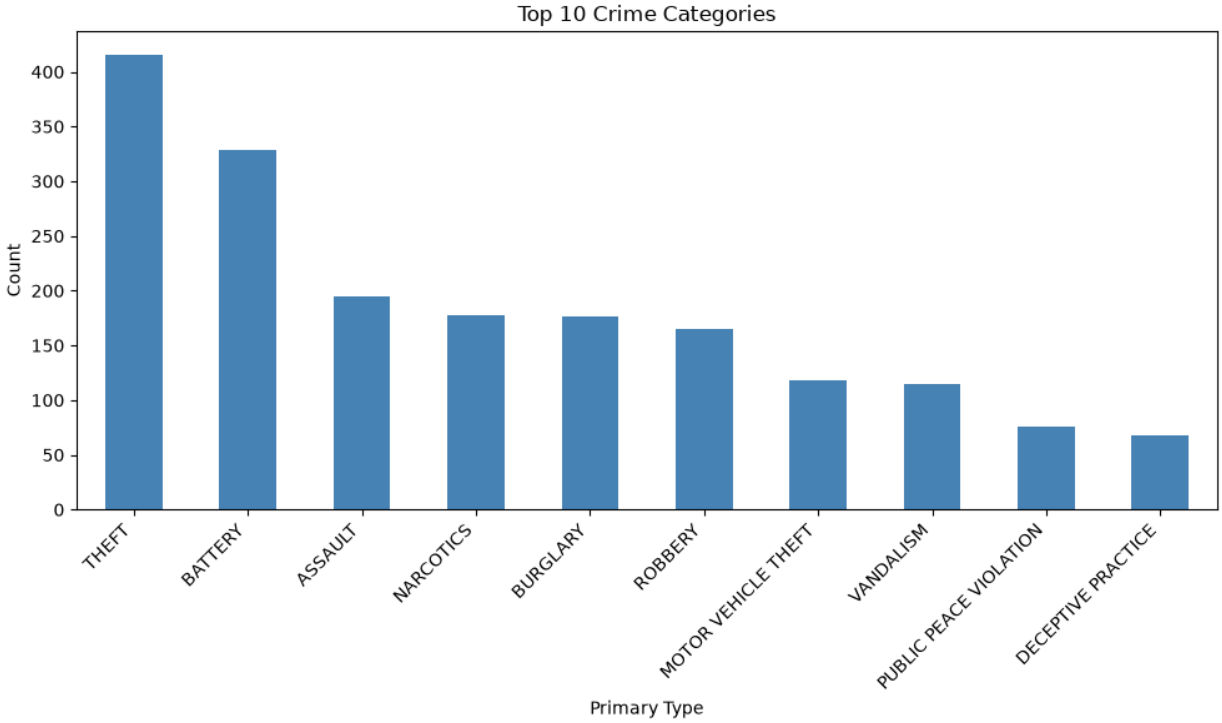
Output

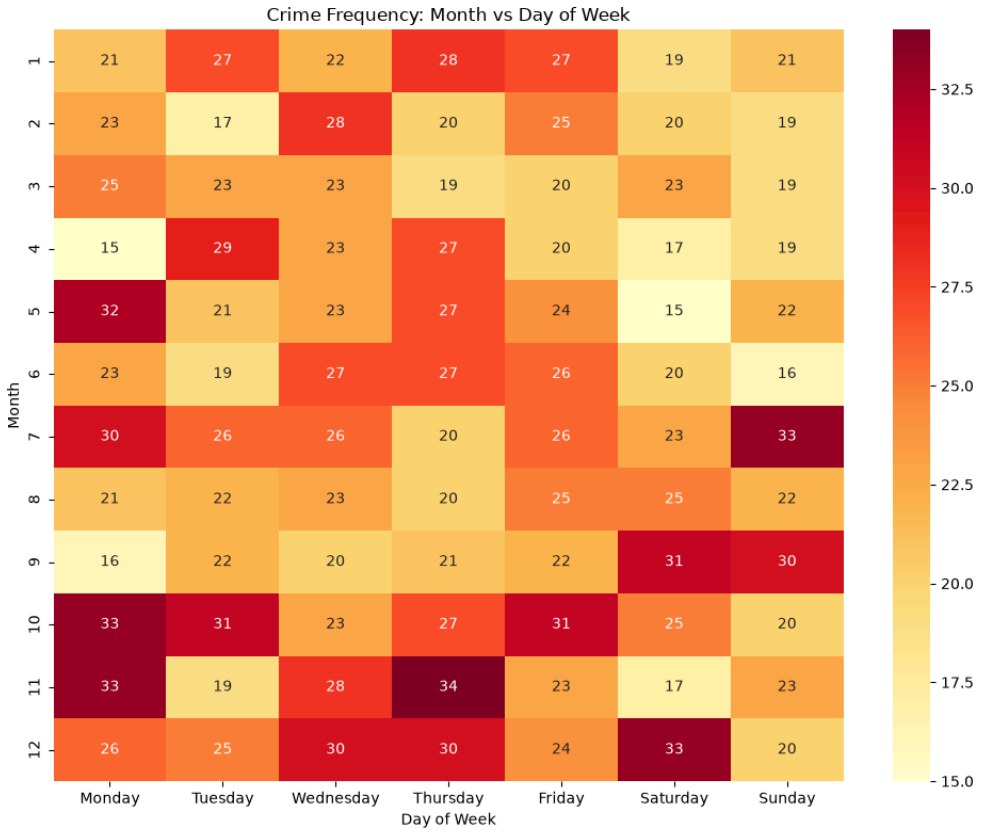
```
1: CRIME TREND OVER YEARS
year
2015    227
2016    242
2017    192
2018    213
2019    236
2020    224
2021    229
2022    221
2023    216
dtype: int64

2: CRIME DISTRIBUTION BY CATEGORY
Counts:
 primary_type
THEFT                   416
BATTERY                 328
ASSAULT                 195
NARCOTICS               178
BURGLARY                177
ROBBERY                 165
MOTOR VEHICLE THEFT     118
VANDALISM               115
PUBLIC PEACE VIOLATION   76
DECEPTIVE PRACTICE       68
Name: count, dtype: int64
```

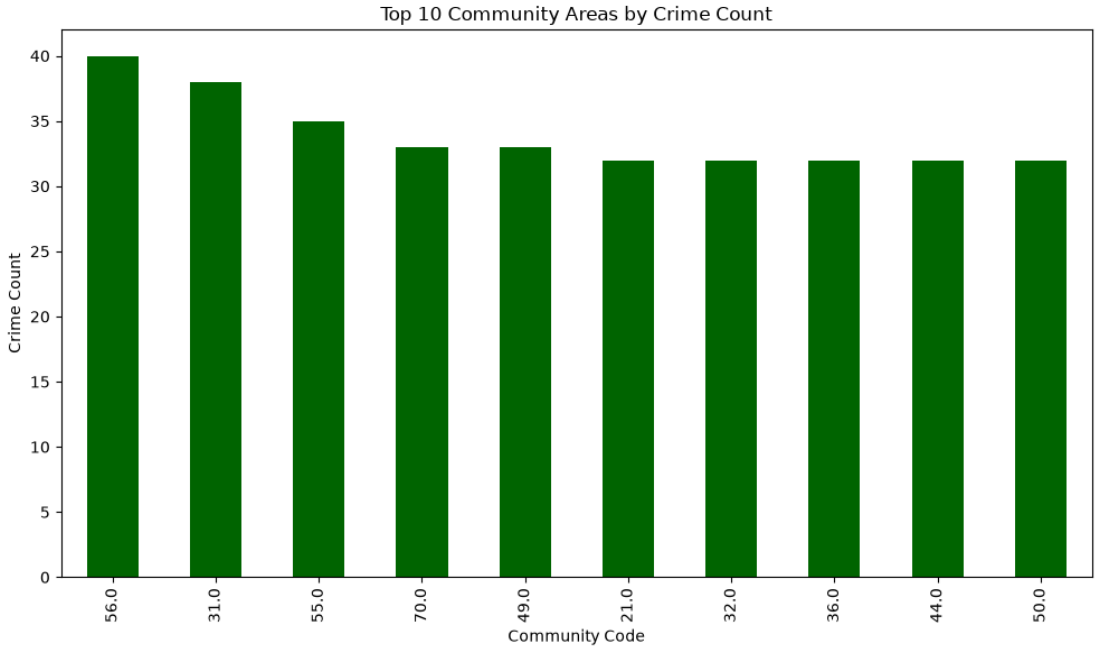
```
Percentages:
 primary_type
THEFT                      20.80
BATTERY                    16.40
ASSAULT                     9.75
NARCOTICS                   8.90
BURGLARY                    8.85
ROBBERY                     8.25
MOTOR VEHICLE THEFT         5.90
VANDALISM                   5.75
PUBLIC PEACE VIOLATION      3.80
DECEPTIVE PRACTICE          3.40
Name: count, dtype: float64

3: ARRESTS AND CRIME OUTCOMES
Overall arrest rate: 29.80%

4: HEATMAP - MONTH VS DAY OF WEEK
Heatmap saved.
```

```
5: TOP COMMUNITY AREAS
community_code
56.0    40
31.0    38
55.0    35
70.0    33
49.0    33
21.0    32
32.0    32
36.0    32
44.0    32
50.0    32
Name: count, dtype: int64

USE CASE 2 — FINAL ANSWERS
Most frequent crime category: THEFT

Arrest rate by year (consistency check):
 year
2015    27.75
2016    25.62
2017    29.17
2018    33.80
2019    31.78
2020    30.36
2021    29.69
2022    32.58
2023    27.78
Name: arrest, dtype: float64

Month with highest crime frequency: 10 with 190 crimes
```


Total Crimes Per Year
Number of Crimes
240
230
220
210
200
190
2015
2016
2017
2018
2019
2020
2021
2022
2023
Year


Top 10 Crime Categories
Count
400
350
300
250
200
150
100
50
0
THEFT
BATTERY
ASSAULT
NARCOTICS
BURGLARY
ROBBERY
MOTOR VEHICLE THEFT
VANDALISM
PUBLIC PEACE VIOLATION
DECEPTIVE PRACTICE
Primary Type


Crime Frequency: Month vs Day of Week
Month
Day of Week
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
1 21 27 22 28 27 19 21
2 23 17 28 20 25 20 19
3 25 23 23 19 20 23 19
4 15 29 23 27 20 17 19
5 32 21 23 27 24 15 22
6 23 19 27 27 26 20 16
7 30 26 26 20 26 23 33
8 21 22 23 20 25 25 22
9 16 22 20 21 22 31 30
10 33 31 23 27 31 25 20
11 33 19 28 34 23 17 23
12 26 25 30 30 24 33 20
32.5
30.0
27.5
25.0
22.5
20.0
17.5
15.0


Top 10 Community Areas by Crime Count
Crime Count
Community Code
0
5
10
15
20
25
30
35
40
56.0
31.0
55.0
70.0
49.0
21.0
32.0
36.0
44.0
50.0

Use Case 3 - Statistical Insights

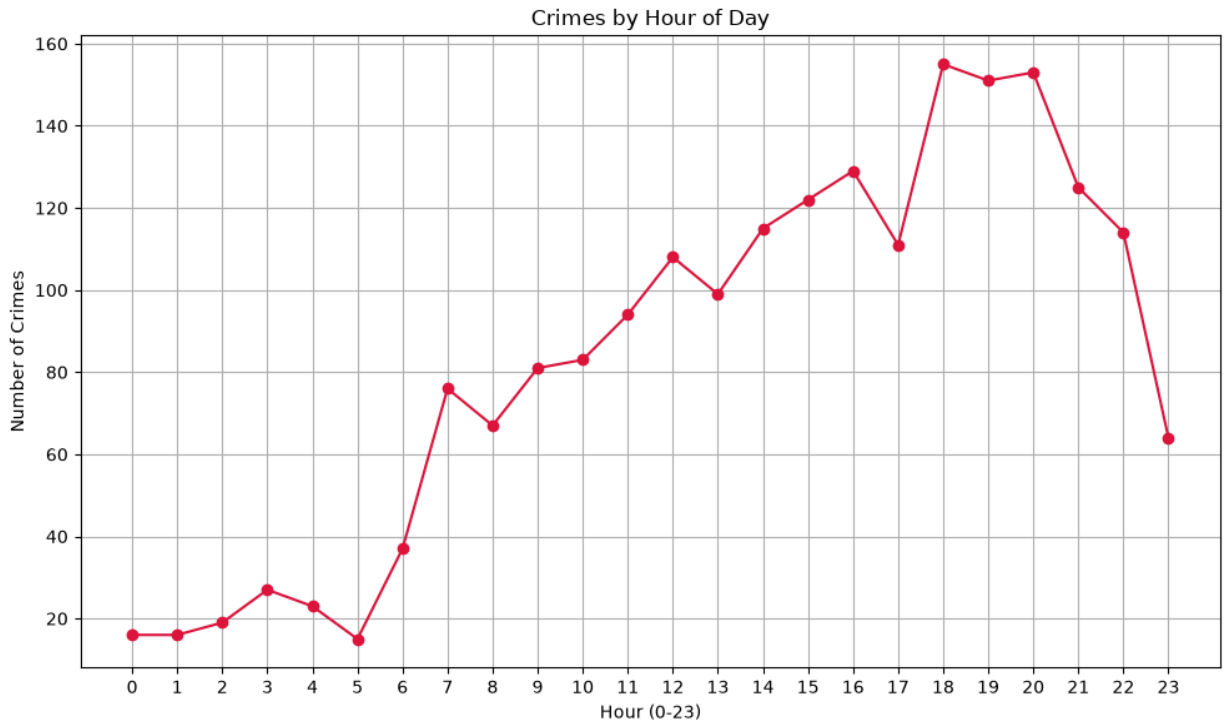
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "..", "data", "chicago_crime.db")
OUTPUTS_DIR = os.path.join(BASE_DIR, "..", "outputs")

os.makedirs(OUTPUTS_DIR, exist_ok=True)

conn = sqlite3.connect(DB_PATH)
df = pd.read_sql("SELECT * FROM chicago_crime", conn)
conn.close()

df['date'] = pd.to_datetime(df['date'])

#1: Crime Intensity by Time
print("1: CRIME INTENSITY BY TIME")
df['Hour'] = df['date'].dt.hour
crimes_by_hour = df.groupby('Hour').size()
plt.figure(figsize=(10, 6))
crimes_by_hour.plot(kind='line', marker='o', color='crimson')
plt.title("Crimes by Hour of Day")
plt.xlabel("Hour (0-23)")
plt.ylabel("Number of Crimes")
plt.grid(True)
plt.xticks(range(0, 24))
plt.tight_layout()
plt.savefig(os.path.join(OUTPUTS_DIR, "11_crimes_by_hour.png"))
plt.close()
print(crimes_by_hour)
```

```
#2: Community Area Clusters Using NumPy
print("\n2: COMMUNITY AREA CLUSTERS (NUMPY)")
mean_per_area = df.groupby('community_code').size()
plt.figure(figsize=(8, 6))
plt.boxplot(mean_per_area.values)
plt.title("Crime Count Distribution Across Community Areas")
plt.ylabel("Crime Count")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUTS_DIR, "12_community_area_boxplot.png"))
plt.close()

Q1 = np.percentile(mean_per_area, 25)
Q3 = np.percentile(mean_per_area, 75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = mean_per_area[(mean_per_area < lower_bound) | (mean_per_area > upper_bound)]
print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
print(f"Outlier bounds: [{lower_bound:.2f}, {upper_bound:.2f}]")
print("\nOutlier community areas (extreme crime counts):\n", outliers)

#3: Crime Cross-Correlation
print("\n3: CRIME CROSS-CORRELATION")
numeric_df = df[['year', 'crime_month', 'arrest', 'domestic']].copy()
numeric_df['arrest'] = numeric_df['arrest'].astype(int)
numeric_df['domestic'] = numeric_df['domestic'].astype(int)
corr = numeric_df.corr()
print(corr)
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap='coolwarm', center=0)
plt.title("Correlation Matrix: Year, Month, Arrest, Domestic")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUTS_DIR, "13_correlation_matrix.png"))
plt.close()
print("\nCorrelation heatmap saved.")
```

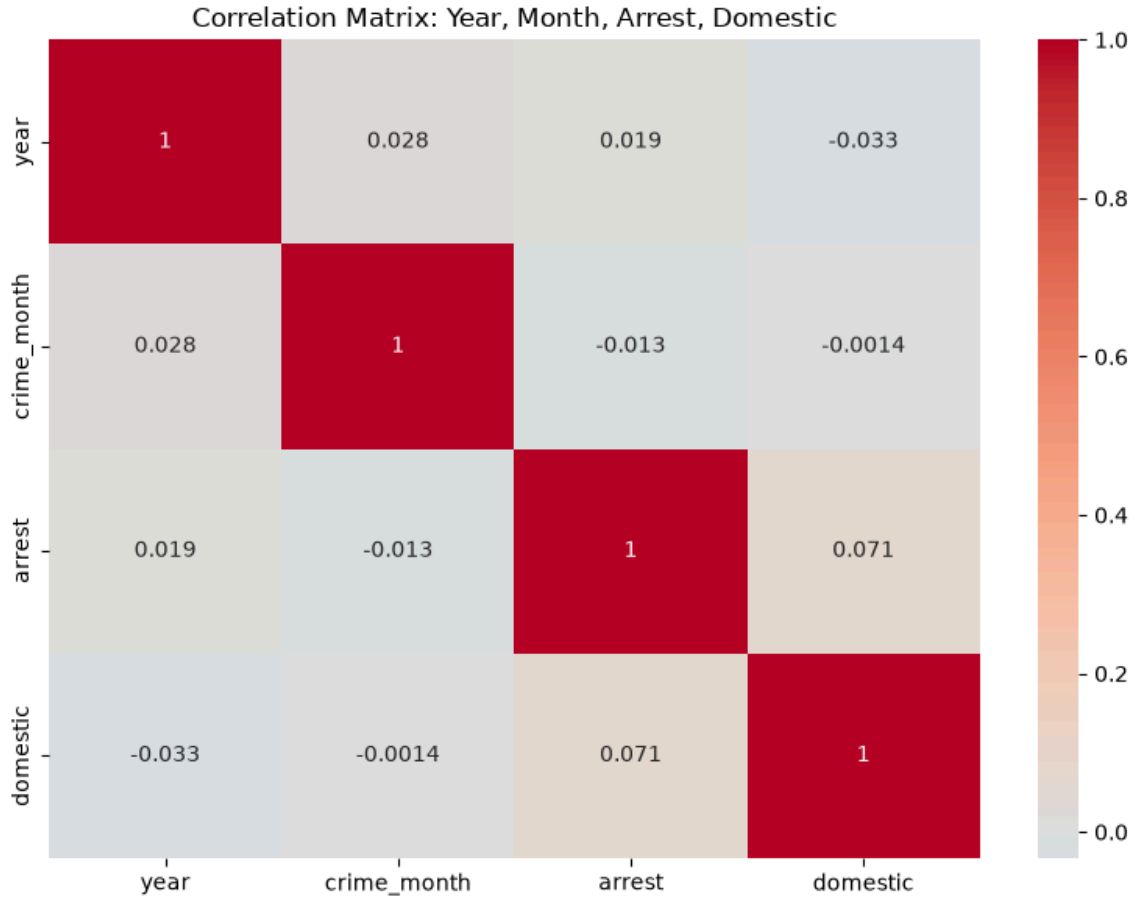
Output

```
1: CRIME INTENSITY BY TIME
Hour
0       16
1       16
2       19
3       27
4       23
5       15
6       37
7       76
8       67
9       81
10      83
11      94
12     108
13      99
14     115
15     122
16     129
17     111
18     155
19     151
20     153
21     125
22     114
23      64
dtype: int64
```

```
2: COMMUNITY AREA CLUSTERS (NUMPY)
Q1: 22.0, Q3: 29.0, IQR: 7.0
Outlier bounds: [11.50, 39.50]

Outlier community areas (extreme crime counts):
 community_code
56.0    40
dtype: int64

3: CRIME CROSS-CORRELATION
                year  crime_month    arrest  domestic
year        1.000000     0.027701  0.018835 -0.033009
crime_month 0.027701     1.000000 -0.013193 -0.001371
arrest      0.018835    -0.013193  1.000000  0.070800
domestic   -0.033009    -0.001371  0.070800  1.000000

Correlation heatmap saved.
```




Crime Count Distribution Across Community Areas
Crime Count
40
35
30
25
20
15
1


Correlation Matrix: Year, Month, Arrest, Domestic
year
crime_month
arrest
domestic
1
0.028
0.019
-0.033
0.028
1
-0.013
-0.0014
0.019
-0.013
1
0.071
-0.033
-0.0014
0.071
1
year
crime_month
arrest
domestic
1.0
0.8
0.6
0.4
0.2
0.0

Use Case 4 - MySQL/SQLite Reporting

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "..", "data", "chicago_crime.db")
OUTPUTS_DIR = os.path.join(BASE_DIR, "..", "outputs")

os.makedirs(OUTPUTS_DIR, exist_ok=True)

conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
```

```
#2: SQL Queries
print("2: SQL QUERIES")

print("\n Crime count per year ")
q1 = "SELECT year, COUNT(*) AS crime_count FROM chicago_crime GROUP BY year ORDER BY year;"
df_q1 = pd.read_sql(q1, conn)
print(df_q1)

print("\n Top 5 crime types and their percentages ")
q2 = """
SELECT primary_type,
       COUNT(*) AS cnt,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM chicago_crime), 2) AS pct
FROM chicago_crime
GROUP BY primary_type
ORDER BY cnt DESC
LIMIT 5;
"""
df_q2 = pd.read_sql(q2, conn)
print(df_q2)

print("\n Arrest count per year ")
q3 = "SELECT year, SUM(arrest) AS arrests FROM chicago_crime GROUP BY year ORDER BY year;"
df_q3 = pd.read_sql(q3, conn)
print(df_q3)
```

```
# 3: Database Stored Views
print("\n 3: CREATE VIEWS ")

cursor.execute("DROP VIEW IF EXISTS vw_crime_yearly;")
cursor.execute("""
CREATE VIEW vw_crime_yearly AS
SELECT year, COUNT(*) AS total
FROM chicago_crime
GROUP BY year;
""")

cursor.execute("DROP VIEW IF EXISTS vw_crime_by_category;")
cursor.execute("""
CREATE VIEW vw_crime_by_category AS
SELECT primary_type, COUNT(*) AS total
FROM chicago_crime
GROUP BY primary_type;
""")

conn.commit()
print("Views created: vw_crime_yearly, vw_crime_by_category")
```

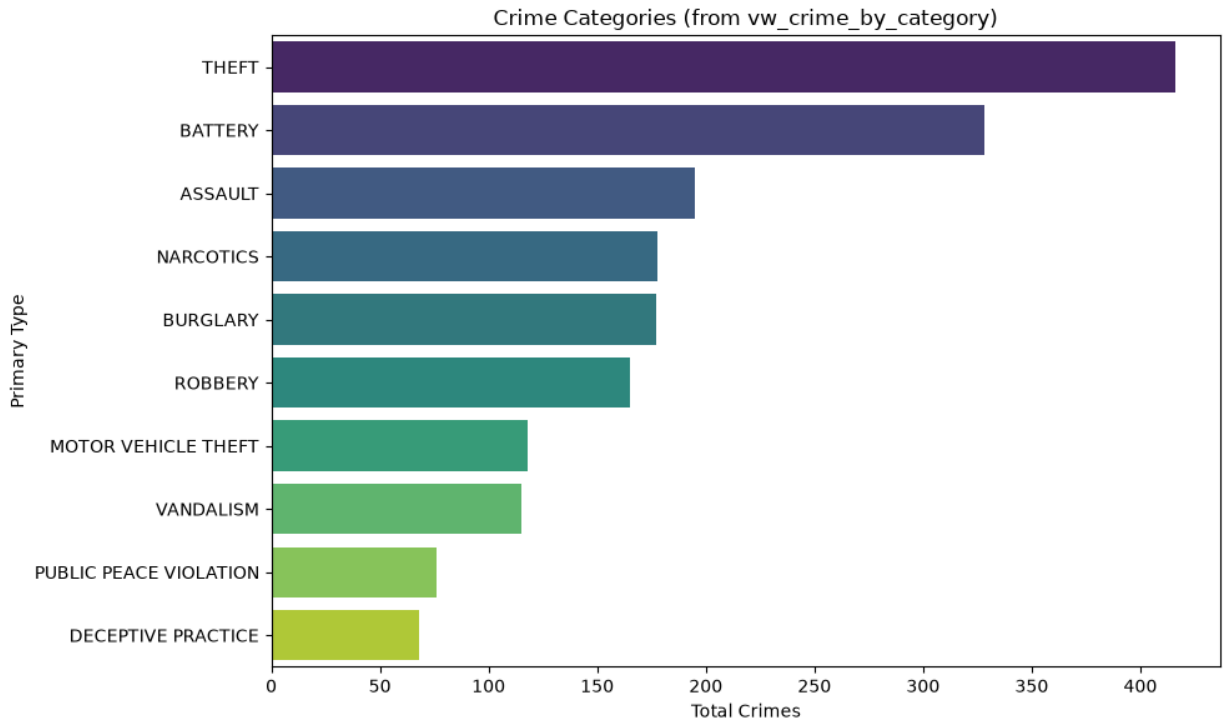
```
#4: Pandas Integration
print("\n=== TASK 4: READ VIEWS INTO PANDAS ===")

yearly_view_df = pd.read_sql("SELECT * FROM vw_crime_yearly", conn)
print("\nvw_crime_yearly:\n", yearly_view_df)

category_view_df = pd.read_sql("SELECT * FROM vw_crime_by_category", conn)
print("\nvw_crime_by_category:\n", category_view_df)

#5: Visualization from SQL Data
print("\n=== TASK 5: VISUALIZATION FROM SQL DATA ===")

plt.figure(figsize=(10, 6))
plt.bar(yearly_view_df['year'].astype(str), yearly_view_df['total'], color='teal')
plt.title("Crime Count Per Year (from vw_crime_yearly)")
plt.xlabel("Year")
plt.ylabel("Total Crimes")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUTS_DIR, "05_sql_crime_per_year.png"))
plt.close()

plt.figure(figsize=(10, 6))
top_categories = category_view_df.sort_values('total', ascending=False).head(10)
sns.barplot(data=top_categories, x='total', y='primary_type', palette='viridis')
plt.title("Crime Categories (from vw_crime_by_category)")
plt.xlabel("Total Crimes")
plt.ylabel("Primary Type")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUTS_DIR, "06_sql_crime_by_category.png"))
plt.close()

print("Charts saved to", os.path.join(OUTPUTS_DIR, "05_sql_crime_per_year.png"),
      "and", os.path.join(OUTPUTS_DIR, "06_sql_crime_by_category.png"))

conn.close()
```

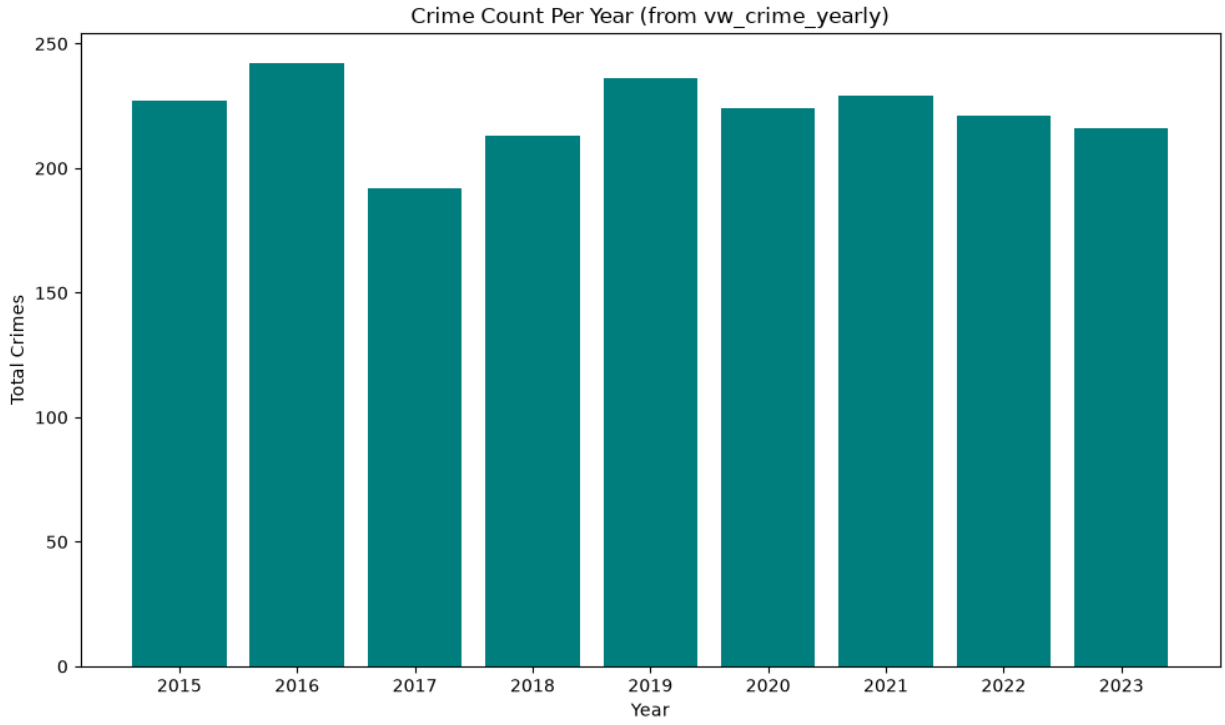
Output

```
2: SQL QUERIES

 Crime count per year
   year  crime_count
0  2015          227
1  2016          242
2  2017          192
3  2018          213
4  2019          236
5  2020          224
6  2021          229
7  2022          221
8  2023          216

 Top 5 crime types and their percentages
  primary_type  cnt    pct
0        THEFT  416  20.80
1      BATTERY  328  16.40
2      ASSAULT  195   9.75
3    NARCOTICS  178   8.90
4     BURGLARY  177   8.85

 Arrest count per year
   year  arrests
0  2015       63
1  2016       62
2  2017       56
3  2018       72
4  2019       75
5  2020       68
6  2021       68
7  2022       72
8  2023       60
```

```
=== TASK 4: READ VIEWS INTO PANDAS ===

vw_crime_yearly:
    year  total
0  2015    227
1  2016    242
2  2017    192
3  2018    213
4  2019    236
5  2020    224
6  2021    229
7  2022    221
8  2023    216

vw_crime_by_category:
               primary_type  total
0                     ARSON     20
1                   ASSAULT    195
2                   BATTERY    328
3                  BURGLARY    177
4       CRIM SEXUAL ASSAULT     13
5         CRIMINAL TRESPASS     50
6        DECEPTIVE PRACTICE     68
7                  HOMICIDE     10
8       MOTOR VEHICLE THEFT    118
9                 NARCOTICS    178
10  PUBLIC PEACE VIOLATION     76
11                  ROBBERY    165
12             SEX OFFENSE     21
13                   THEFT    416
14               VANDALISM    115
15        WEAPONS VIOLATION     50
```

```
=== TASK 5: VISUALIZATION FROM SQL DATA ===

Warning (from warnings module):
  File "C:\Users\Saraswati\OneDrive\Desktop\crime-analytics-capstone\notebooks\06_sql_reporting.py", line 87
    sns.barplot(data=top_categories, x='total', y='primary_type', palette='viridis')
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` an
d set `legend=False` for the same effect.

Charts saved to C:\Users\Saraswati\OneDrive\Desktop\crime-analytics-capstone\notebooks\..\outputs\05_sql_crime_per_year.png
and C:\Users\Saraswati\OneDrive\Desktop\crime-analytics-capstone\notebooks\..\outputs\06_sql_crime_by_category.png
```


Crime Categories (from vw_crime_by_category)
THEFT
BATTERY
ASSAULT
NARCOTICS
BURGLARY
ROBBERY
MOTOR VEHICLE THEFT
VANDALISM
PUBLIC PEACE VIOLATION
DECEPTIVE PRACTICE
Primary Type
0
50
100
150
200
250
300
350
400
Total Crimes


Crime Count Per Year (from vw_crime_yearly)
250
200
150
100
50
0
Total Crimes
2015
2016
2017
2018
2019
2020
2021
2022
2023
Year